INDIAN INSTITUTE OF TECHNOLOGY MADRAS

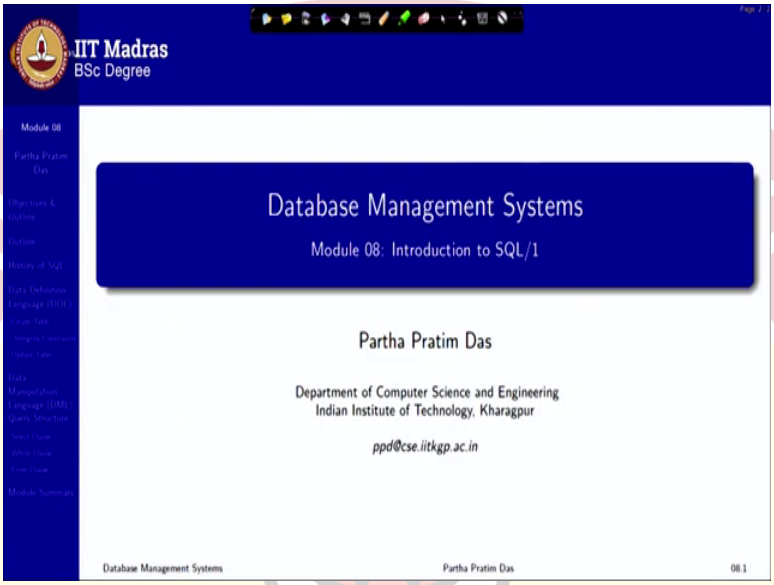
सिद्धिर्भवति कर्मजा

IIT Madras

ONLINE DEGREE

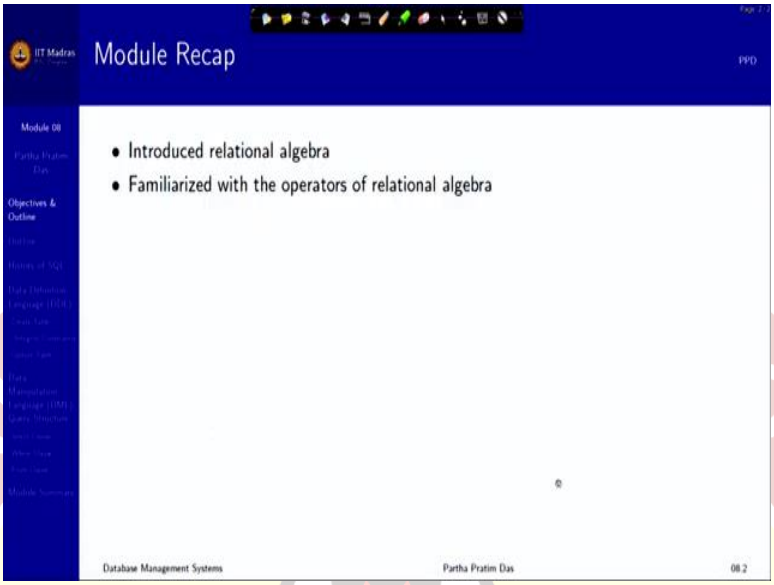
Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Introduction to SQL/1

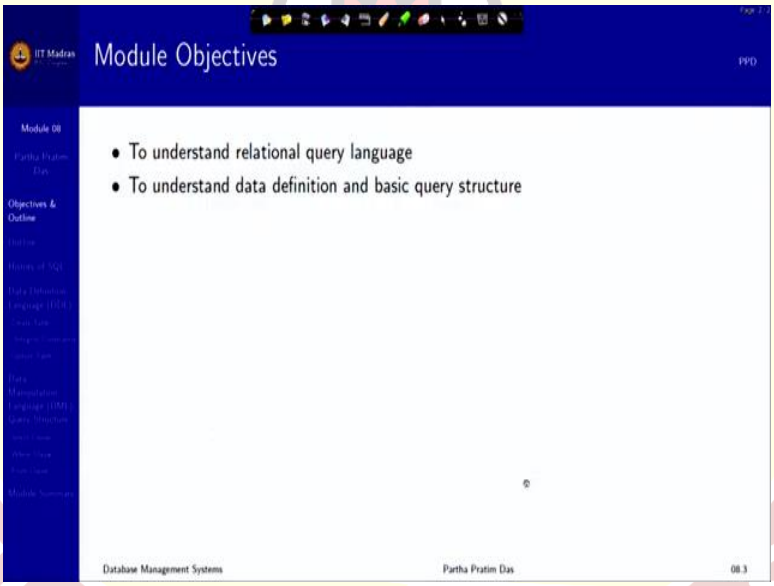
(Refer Slide Time: 0:31)

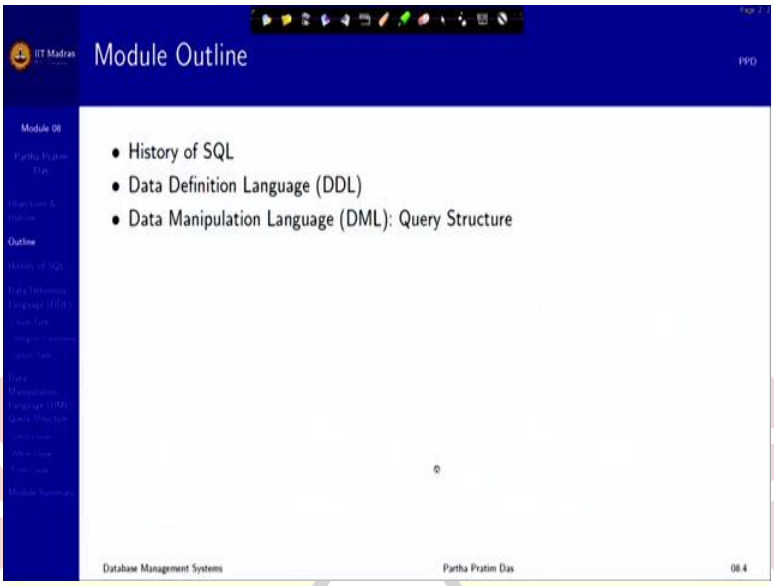


Welcome to Module 8 of database management systems in IIT Madras online BSc program. We have prior to this module given a basic overview of the relational system, we have defined the basic terms like what is an attribute, what is a key, what are different types of keys like super key, candidate key, foreign key and so, on the schema instance and the basic idea that there are two parts in the language that will be used to manipulate any database application system, the Data Definition Language DDL and Data Manipulation Language DML. The language sequel some call it SQL but it is originally sequel. The language sequel can do both of that.

(Refer Slide Time: 1:35)

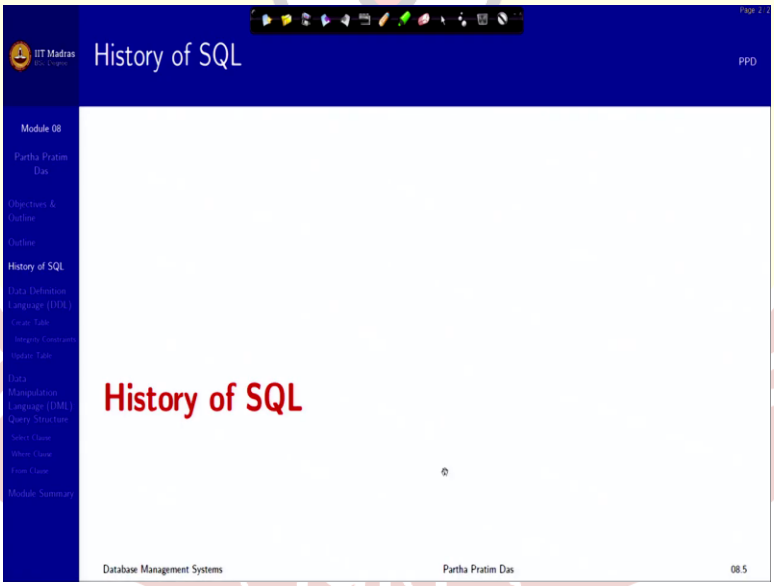


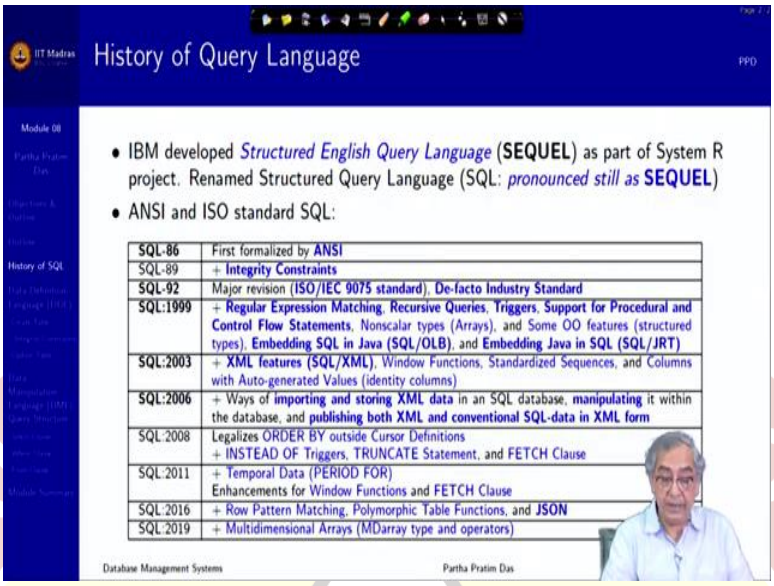




So, in the perspective of what we had done earlier, we will now build up to see this relational query language particularly SQL in greater depth and see its basic query structure. So, these will be the three main topics we will discuss, history, DDL and DML.

(Refer Slide Time: 2:01)





So, first the history, I mean I always prefer to have a look at the past to understand where we are and where we are going. So, structured English query language is how it started. It was started by IBM in their San Jose Research Laboratory as a part of system our project that was in the late 70s, early 80s and so on.

Subsequently it was renamed as Structured Query Language. So, the original name SQL or sequel was retained. Then naturally as you know every language which needs to be used across the world need to be standardized. So, several standardization efforts started first the SQL 86 which was made NC that is it was made an American Standard, American National Standards Institute.

In 89, integrity constraints were added to that, but the major thing happened in 92, SQL 92 when the first international standard ISO standard as we say 9075 was released and till date we are going with that standard making revisions one after the other and once this international standardization happened naturally it started becoming a de facto industry standard.

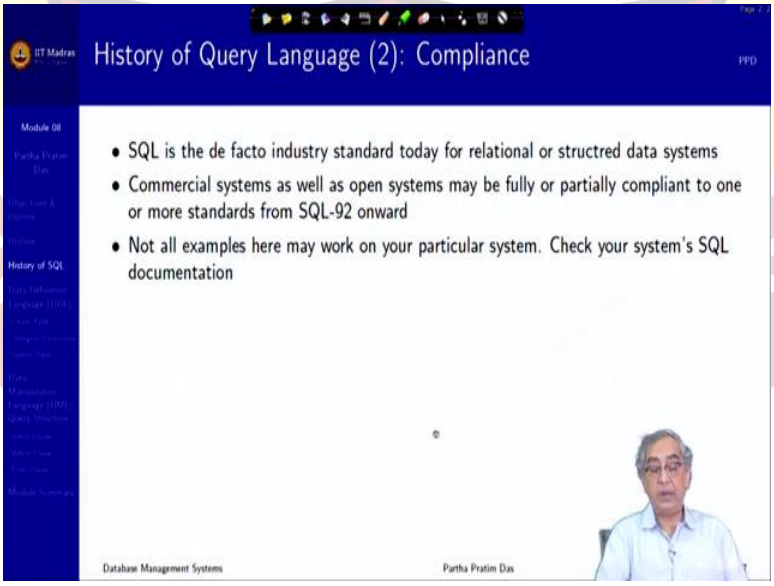
So, what you see in this list, and you will see that I have boldfaced some of the version numbers because they have had more impact or they have been, they have added more fundamental features to the language, naturally SQL 1999 you can understand that this was getting close to year 2000. So, everybody was concerned with y2k. So, the naming of the standard also changed in the y2k four-digit year format.

This added several of very useful features like matching of regular expressions, recursive queries, that is which we will see is not possible in pure relational algebra that is transitive closure was provided as a part of SQL then triggers, support for procedural and control statements, which are SQL as I mentioned, or query languages, as I mentioned are declarative, but some procedural support is also provided.

Arrays, structure types and most importantly, first a formal embedding mechanism was given in SQL, so that you can embed SQL queries in Java, which is called SQL will be and you can embed Java in SQL, which is called SQL JRT. Both of these embeddings are possible so that you can choose which part of the task you would like to do as relational operations or database operations and which part you would like to do in a standard programming language like Java.

2003 standard also added a very significant feature of XML. We will talk about XML being a standard format for Information Interchange. So, that was added in 2003 and in 2006, a full functionality of XML including, importing, storing, manipulating, exporting, publishing, all these were added. Subsequently several small but important updates continued to happen in SQL and the latest version of SQL is 2019.

(Refer Slide Time: 5:54)

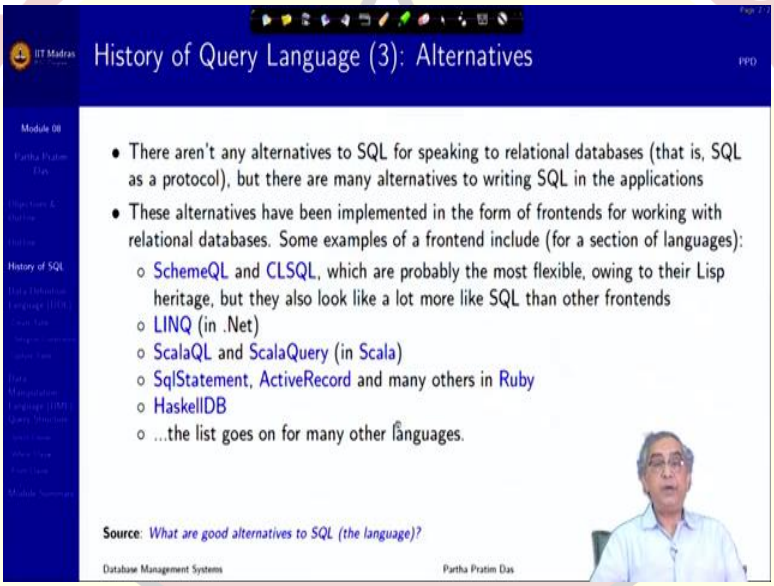


Now, if we look at the compliance as to who is, how widely it is used, it is like, it is kind of the de facto standard, I mean, anybody working with relational databases will use SQL, there is no question about that. But the compliance to the standard is not uniform, some are compliant to a

certain standard fully or maybe multiple standards in partial way and so on. But most commercial databases at least comply with SQL 92 and many comply with 2003 and 2006.

So, it is possible and there are variations in terms of the actual keywords or actual forms in which certain things are expressed in the language. So, not all examples that we are providing here may work in your particular system. I mean this will be true for not only for this course, but in your work life as well. You will have to really check-up the SQL system documentation of the particular SQL release that you are using, this is true for commercial as well as open systems.

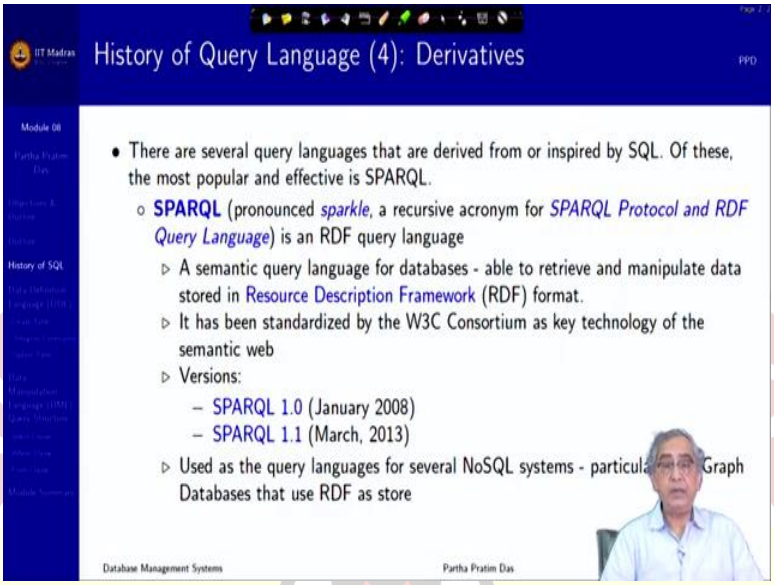
(Refer Slide Time: 7:04)



Now, the question is, are there alternatives to SQL? Like you have in the programming language domain, you have several alternatives you can use C plus plus or you can use Java, you can use Python, so you decide what is your application context and choose the language. But incidentally, in case of relational databases, there is practically no alternative to SQL, there have been some languages like QUEL, Q U E L which as we will see is kind of supports relational calculus kind of queries.

But practically SQL reigns everywhere, only thing is there have been several front ends made which are, which make it easy to write SQL in different languages like Lisp like languages or in Dot Net or in Ruby or in Haskell or in Scala and so on. So, these are mostly the front-end part of the extension, but really not alternatives that way.

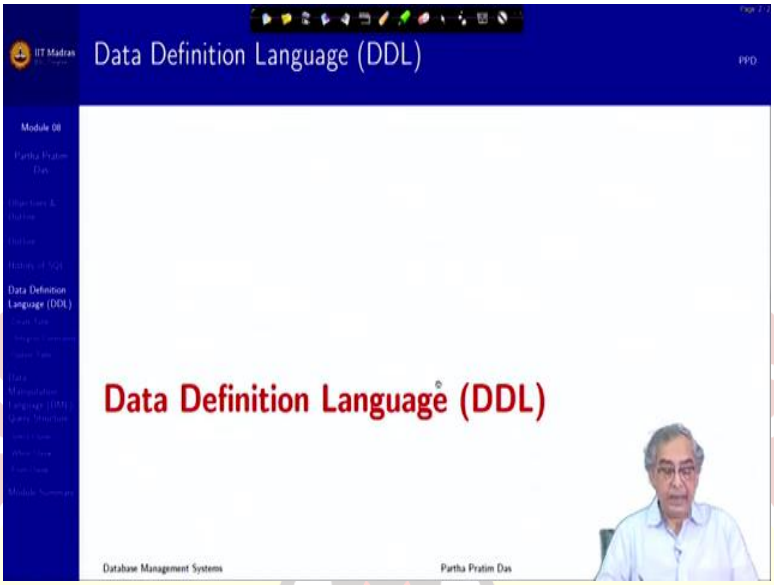
(Refer Slide Time: 8:11)



Now, the other question is, are there derivatives of this language that what other languages been built using SQL as a primary language or the inspiration? Yes, there are many, I mean, it will be over 100. But what stands out today of these are what is known as sparkle. It is written as SPARQL. And it is a kind of funny acronym where the acronym is recursive.

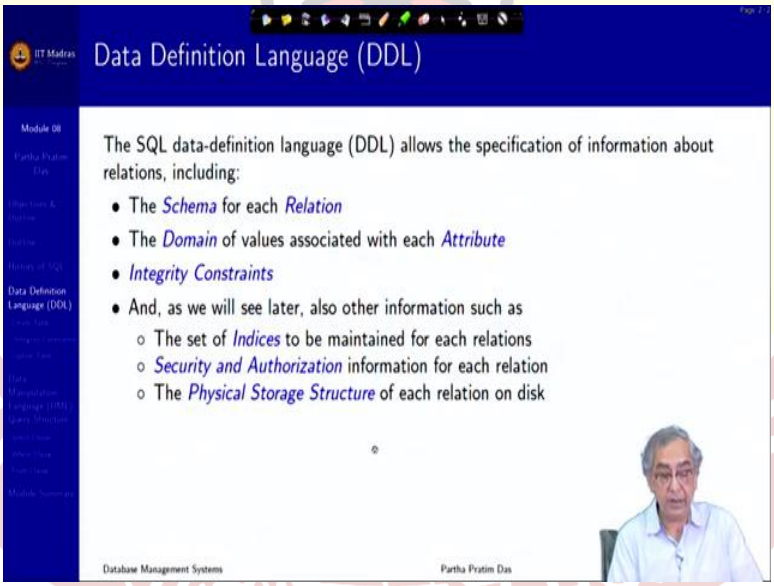
So, sparkle stands for sparkle protocol, and RDF query language. Very interestingly, it is a query language, which is based on not exactly the relational form, but it is an RDF recursive, Resource Description Framework, which is a triple or called the triple structure, which is more generic, but maybe a little bit less efficient. And it was been standardized by W3C, and currently the version is sparkle 1.1 particularly working with several non-SQL system particularly the graph databases, that use RDF as a store, sparkle has become the de facto standard kind of.

(Refer Slide Time: 9:26)



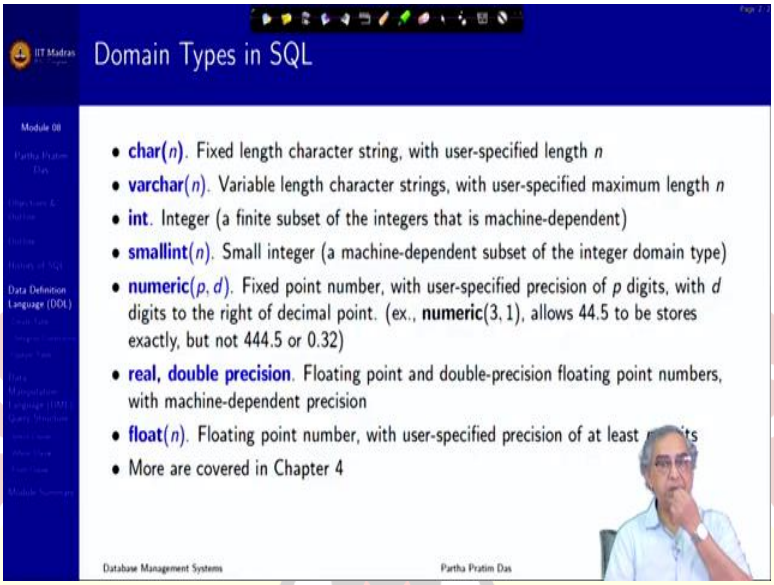
Okay having, had a look at the history let us go into the meet of the language.

(Refer Slide Time: 9:31)



First the data definition, naturally we need to define the schema, we need to define relation, domains of attributes and the integrity constraints. And there are several other definitions that are possible which we will talk about in a little later once we have understood the basic structure, the initial structure.

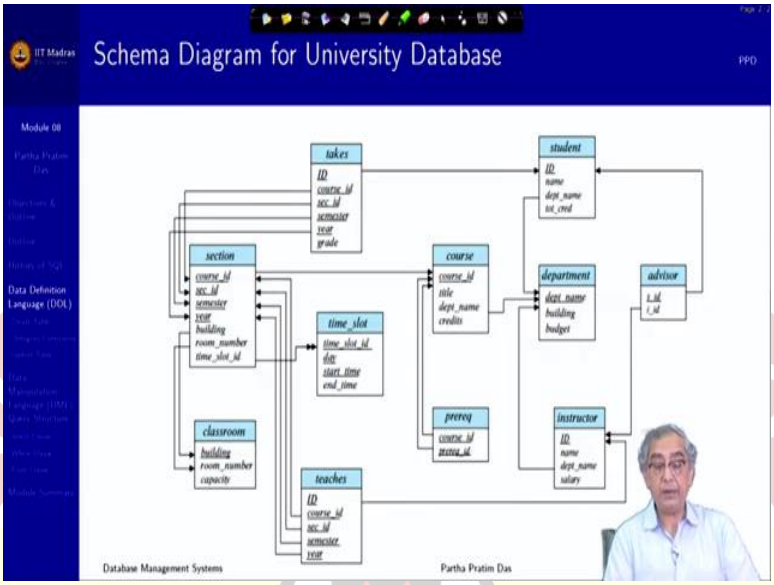
(Refer Slide Time: 9:54)



So, there are several data types, like when you studied Python or C or C plus plus that there are data types. So, there are several data types here, character, var character which is variable character, so char n, I will just mentioned one to char n means that there is a field where n number of characters will be there. Say, if say char five there will be five characters, but varchar is variable which says if I have varchar five, then there could be multiple characters up to n up to five. So, there could be 3 characters also is valid, in char five, three character strings or not valid.

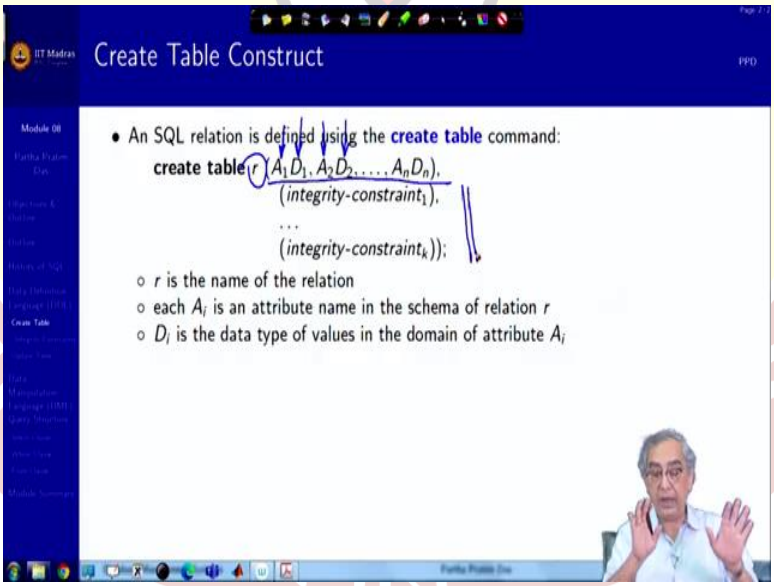
There is integer, small integer, there is specific precision defined numeric, which says what is the total size and what is the number of digits after the decimal point and so on, naturally a floating point and all those. You also have string, which is very, very widely used.

(Refer Slide Time: 10:55)



So, this is a schematic of the university database, which I had requested you to study, I hope you have studied. So, we will use some of these schema tables for creating the definition.

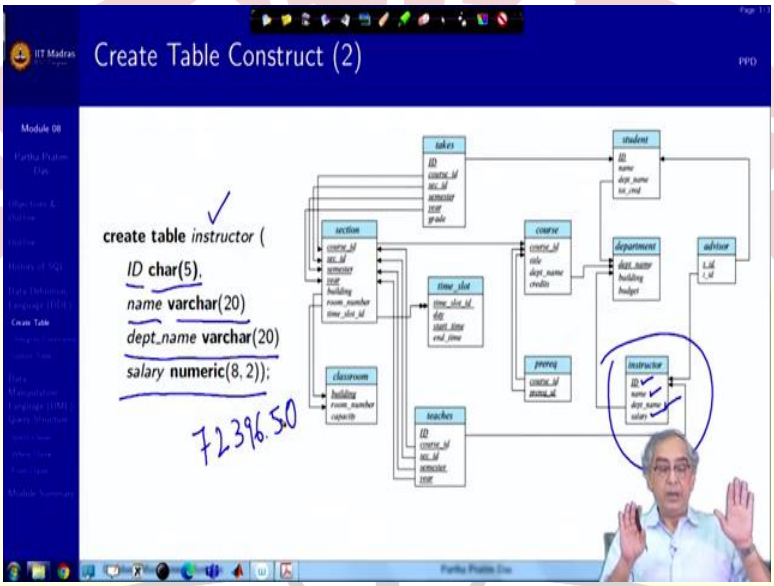
(Refer Slide Time: 11:14)



So, first, how do you define a schema, how do you define a table, all that what, all do you have to do, you will have to give it a name clearly, and you will have to define what are the attributes and what in what order the attributes you want to specify. So, clearly, it becomes that you have to give a name and the attributes.

And the attributes must also have their type specified. So, you can see this is the name of the attribute. And this is the type of that tribute, name of the second attribute type of the second attribute. So, this is a basic definition. And then what you can have is you can have a set of integrity constraints, what all conditions must satisfy. Let us take an example.

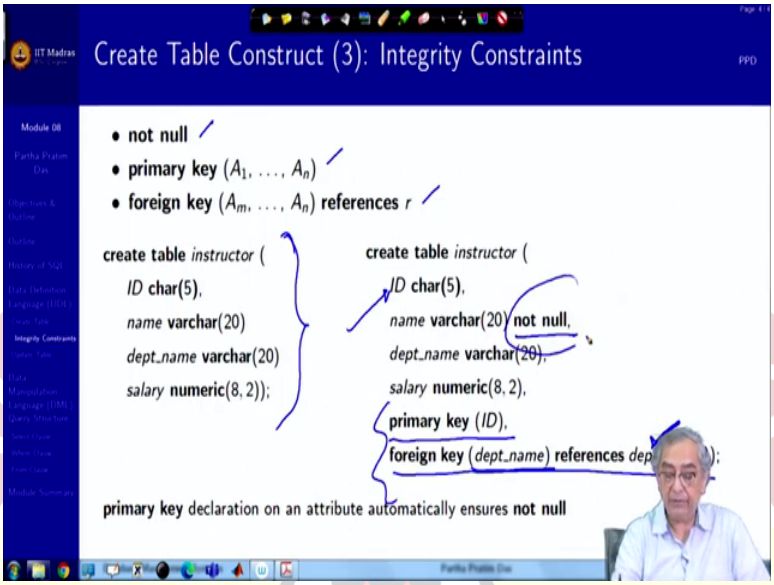
(Refer Slide Time: 12:10)



Say, it is clearly that I am talking about this instructor table. So, it has the name is instructed so I put it here create table instructor. The first attribute is ID, I write it here. And write its type, we had said that the ID will be 5 characters long, so we write char 5. Then the next is the name of the instructor, I put name and keeping varchar 20 because we do not have fixed size name, but so you are putting a maximum length, the maximum length allowed for the name is 20 characters, so varchar 5.

Then the department name, which is also varchar 20. And finally, the salary, which is say is numeric, 8, *2*, which means that the total number of digits in the salary will be 8, and there will be, two maximum will be 8. And there will be up to two decimal point, digits after the decimal point, after the period. So, something like 72396.50 will be a valid value. So, these are the basic definition, the name of the table, attributes and that type.

(Refer Slide Time: 13:31)



Now, that is not all, because there are more properties that we need to say. And this is what we call are integrity constraints, that is we must specify that if there is a field where we will not allow null that is it must be provided for any record that I want to put there. So, it could be just nullity, I want to save what is the primary key because that is very important, as you know, that has to be unique all through.

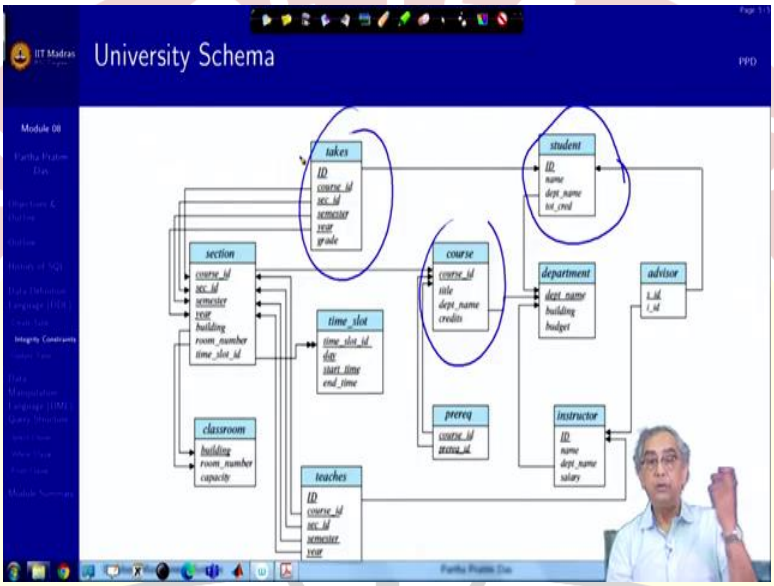
And also want to save there are foreign keys that if there is some field here, which is referred in some other table. So again, this is what we did in the last slide. And this is what we are adding, we say that it is name is not null that is you cannot add an instructor whose name is not specified that is obviously not meaningful.

Then we are saying that primary key ID that is we are mentioning that ID will be the primary key, it will have to be unique. And please remember that the moment you say that primary, something is a primary key, then it is by default, non-null, because certainly having null cannot uniquely find two values.

Then the last thing we say is foreign key department name, references department, because in the department name, we will put the name of a department which must exist in the department table, if the department table does not have say aerospace and I write aerospace in the instructor table, then obviously I will not be able to find the say the building of a particular professor's office who is there in the instructor list.

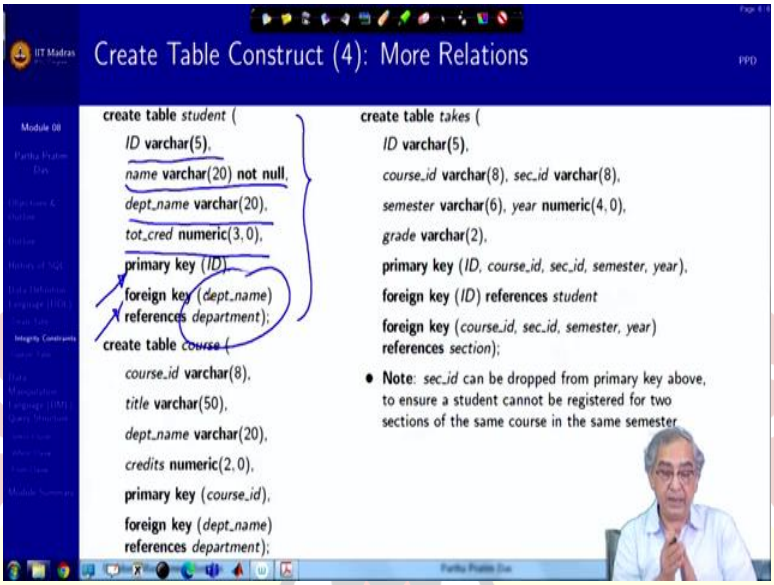
So, this means, this puts the constraint that any department name which is used here must be available in the department table. So, this is how we, so we have specified the table name, the names of the attributes, their order, their types and the constraints. So, here we have three constraints specified.

(Refer Slide Time: 15:54)



So, with that, we can go ahead and just take a look at for example, some of these we will try to do we will try to show how students are done, courses are done, takes are done, takes are what course a particular student takes and so, on.

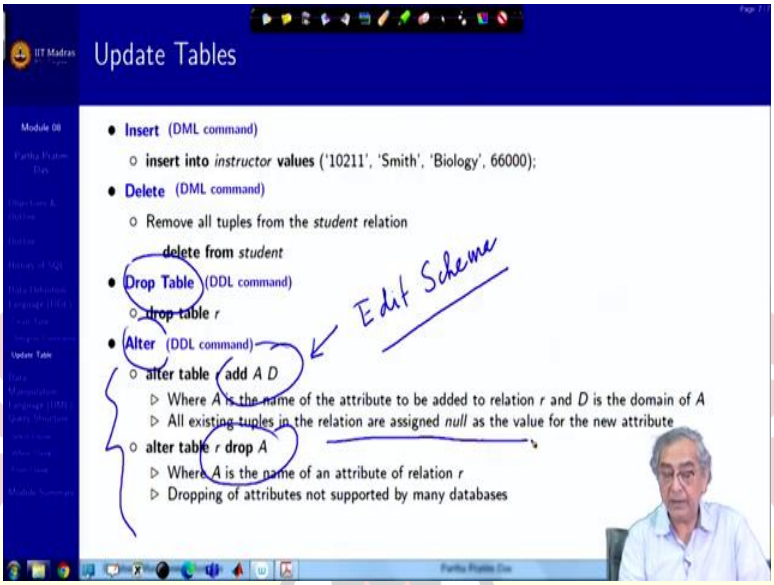
(Refer Slide Time: 16:09)



So, if you look at here, when you study at home, you will have to study them in depth. So, you can see that this is what the student table creation processes. So, you have ID which is varchar, you have made that a primary key so it by default becomes not null then the name is not null, like before department is varchar, it may be known may not be known. So, we are not saying that not null, total credit is numeric 30.

So, it cannot have a decimal part. So, it has to be a whole number what this necessarily means, it is a whole number which is up to 3 digits, the primary key is ID and foreign key is department name like before. So, it is very easy that way, you can go through the rest with reference to the schematic of the table at your study time and convince yourself that or basically try to understand yourself as to why the different types are given as they are and why the different constraints have been given as they are.

(Refer Slide Time: 17:31)



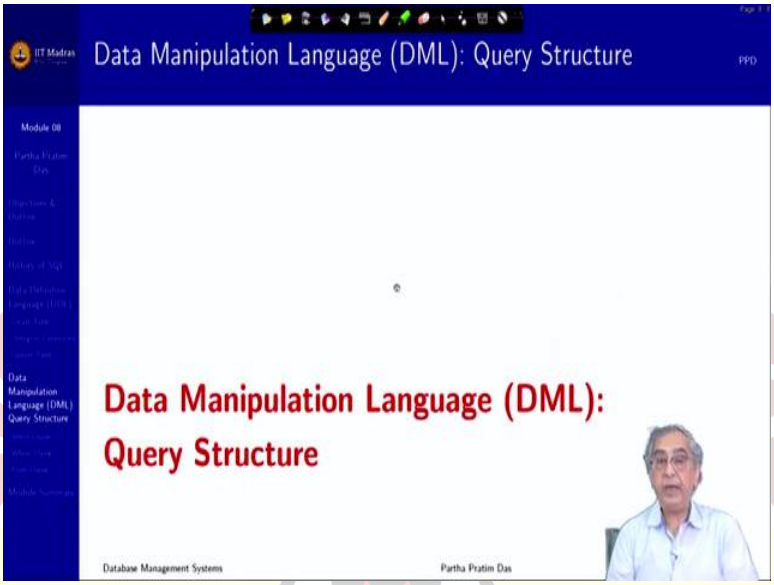
Now, the other operations that you can do is in the table is you can insert a value which is more like a data manipulation operation, but you can still add, we will talk more about insert later on, but what you can also do is you can remove all elements or all rows from a table. So, you can say delete from students. So, you can remove everything.

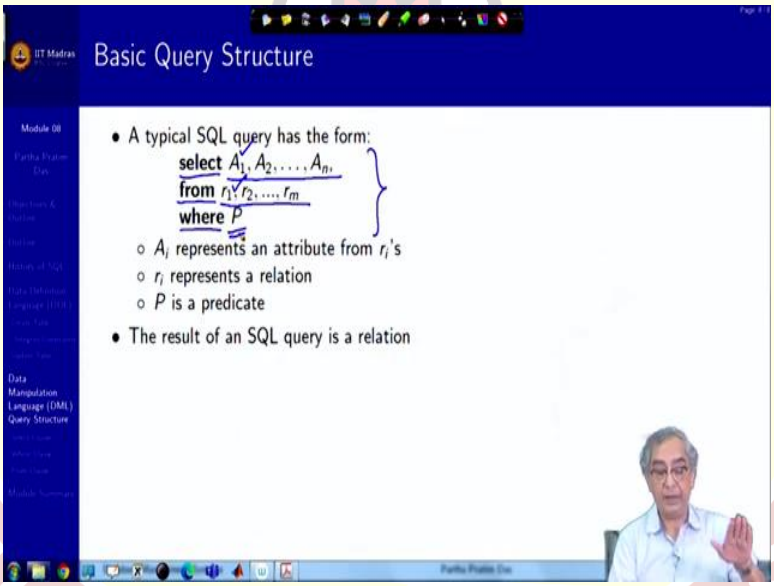
Then you can have a drop table that is you want to discard the table altogether, so that it will be removed from the data dictionary itself or you can alter a table or you can alter a table, add an attribute of a particular data type or drop a particular attribute. So, it is kind of editing the schema. So, these all drop table, alter these all are basically edit schema.

So, first is naturally the, first is naturally the creation process and then the editing process for the tables, which is important naturally, you will understand that if you drop an attribute then all tuples which are already existing in the table will lose the value on that attribute. Similarly, if you add an attribute then it will have to be a nullable attribute to start with, because what values will the different tuples take on that attribute, the values are not there.

So, they will have to do some update on the specific records to put those values. So initially, all values will have to be null. So, they will be assigned null value. So that is very, very intuitive. These are the biggest thing about SQL is once you start understanding the gamut of things being done, this is very intuitive in nature.

(Refer Slide Time: 19:40)





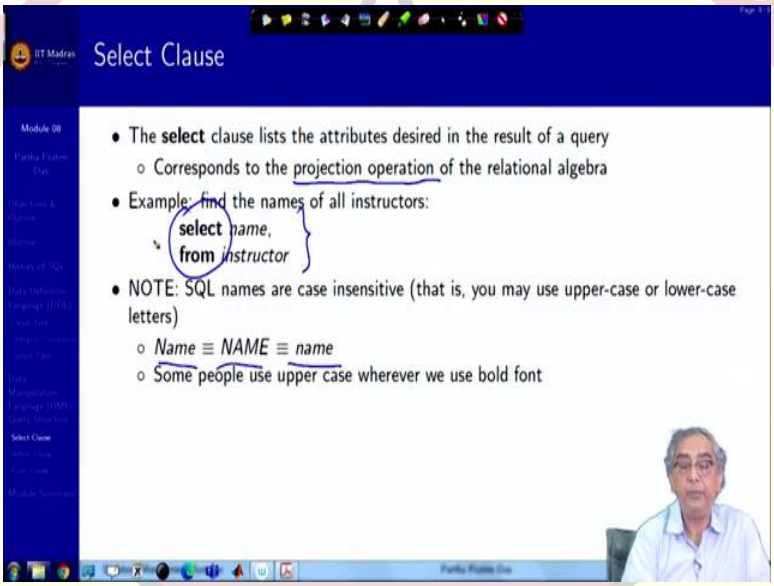
Next, we move on to the data manipulation or query language which is which goes by very simple in SQL it goes by very simple structure and around that basic structure. All different refinements and advances are given. So let us understand this structure first. We often call it a select from where, where you are select from where structure, select from where clause. So, there are three main parts, select, from and where.

Select is a list of attributes that are given. Now, where do these attributes come from? Attributes are not entities by themselves, they are part of certain relation. So, from specifies that scope that what are the relations that exist in the scope of this select from where. So, I have a number of

relations, each relation have number of attributes. So, I am talking about when I am talking about say A1 I am saying that it is from relation R1, it is not necessary to have multiple, but at least it is necessary to have at least one relation.

And from one relation, you may have multiple attributes that you are referring to, they are uniquefite by the name of the attribute. And then finally, in where clause, you have a predicate, predicate is nothing other than a Boolean condition. So, let us take a deeper look into each one of them. We will go into each part of this query clause.

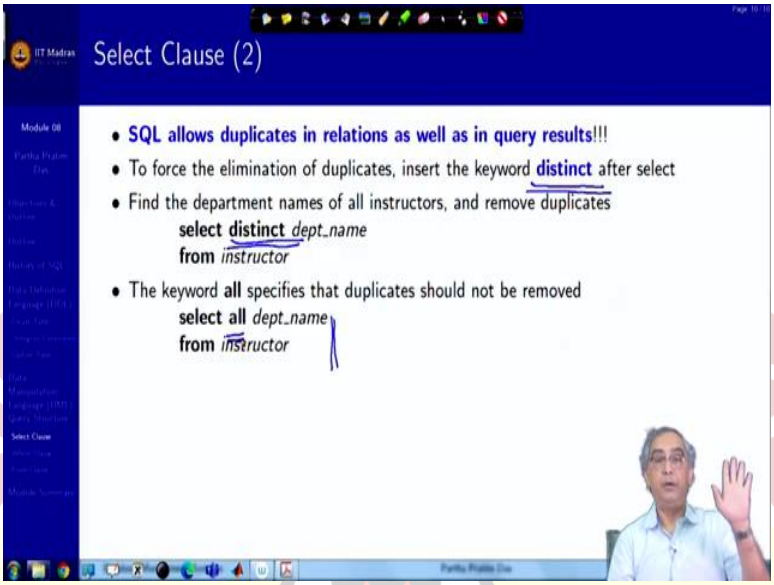
(Refer Slide Time: 21:28)



So, first is a select clause, select clause what it does is it specifies the attributes that I want in the output. So, it is a kind of, it is actually the projection operation of relational algebra that we had discussed. So, I say select name from instructor, so instructor is the relation. So, if I select name from instructor the name has to be an attribute in instructor. So, what it will do, it will take the instructor table, take out the name part of it and give me as an output simple.

It may also be noted that SQL in general is names and all are case insensitive. So, you can write the name as this, as this, as this or any other combination. So, so far as the spelling is the same SQL takes them as the same. Even the SQL keywords could be lowercase or uppercase, we are using boldface to just make it easy to understand. So, this is a select which tells you the projection what all attributes we are taking.

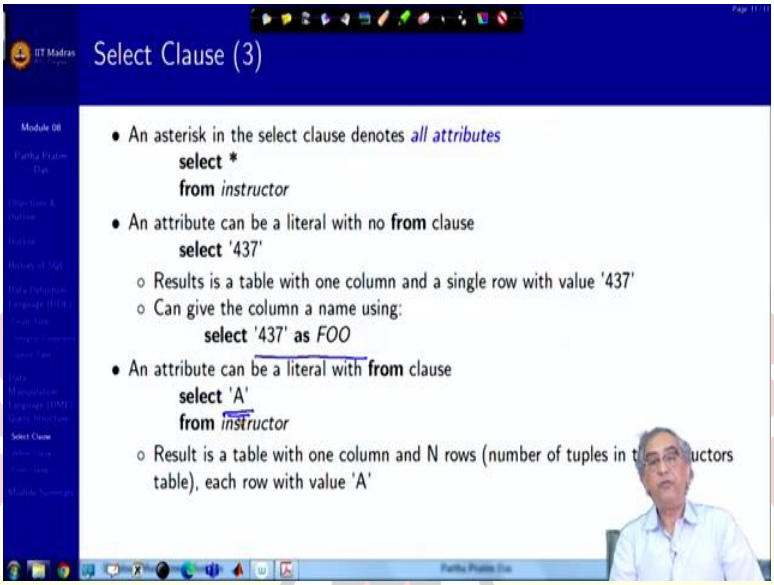
(Refer Slide Time: 22:43)



Now, interesting part in select is SQL we said that the relational algebra is follows a set theory, so it does not allow duplicates, but in SQL actually allow duplicates. So, for example, if name is not a key, then it is possible that you will have duplicate names. So, if you take out the name, there would be multiple entries with the same name and SQL result will actually show that, so SQL by default allows duplicates in relation in the query result as well.

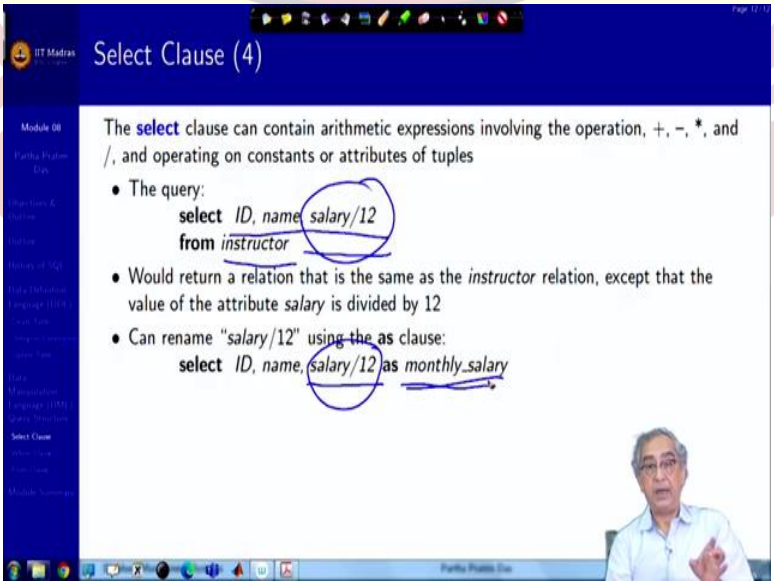
So, if you do not want the duplicates to come, if you want it behave more like the set relational set theoretic result, then you have to say distinct, so it is a select distinct what this will do, it will first select the department name and then make it unique that is if any department name is occurring more than once, it will be made to occur only once. So, and if you want everything without specifies the duplicate should not be removed, I can do it by default. Or you can specify to explicitly say that you need all you can specify all. So, this is.

(Refer Slide Time: 24:09)



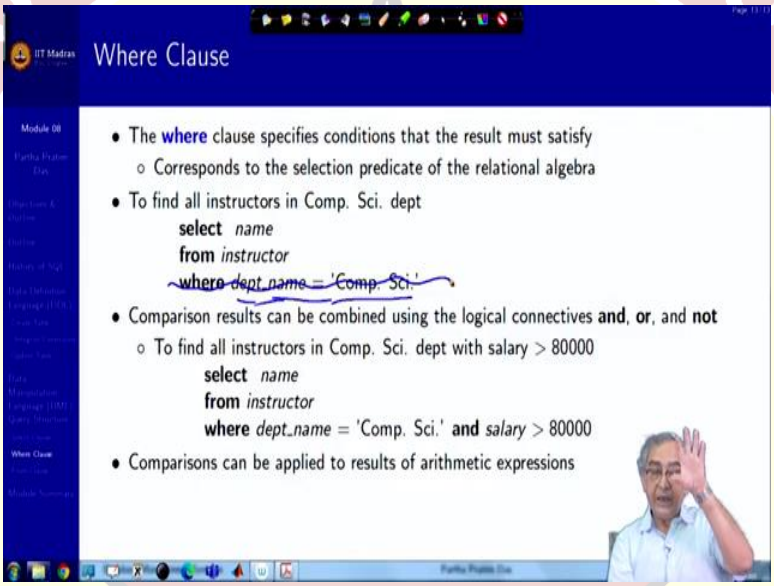
Now, there are certain shortcuts or certain specific forms of select, you can write a wildcard like * which means all attributes of a relation. Or you can specify a literal also. For example, if you say select within quote “423”, it is a string literal. So, it will return you a table containing only one row having that value. You can also provide a name to that, you can say that this FOO, the name of that attribute. So, it is kind of a way by which you can kind of take a value and make it into a table and then use it in other parts of that query. So, it can have a literal from the from clause as well.

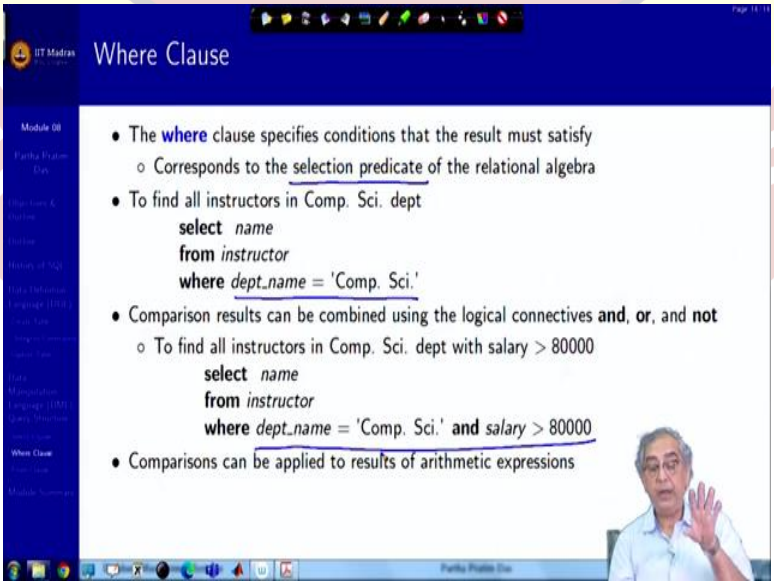
(Refer Slide Time: 25:13)



So, there are there are various nuances and variations of this. For example, here you can do this, look at this, what we are doing, we are taking the instructor table, we have taken the ID, the name and salary we have done salary by 12. So, what will happen the salary column all values will be replaced divided by 12. So, that is the kind of different shortcuts that you can keep on doing. You can also rename that and make it into a new column say monthly salary. So, say salary by 12 so it takes the salary from the salary attribute of instructor and in the output makes a new column month salary.

(Refer Slide Time: 26:10)

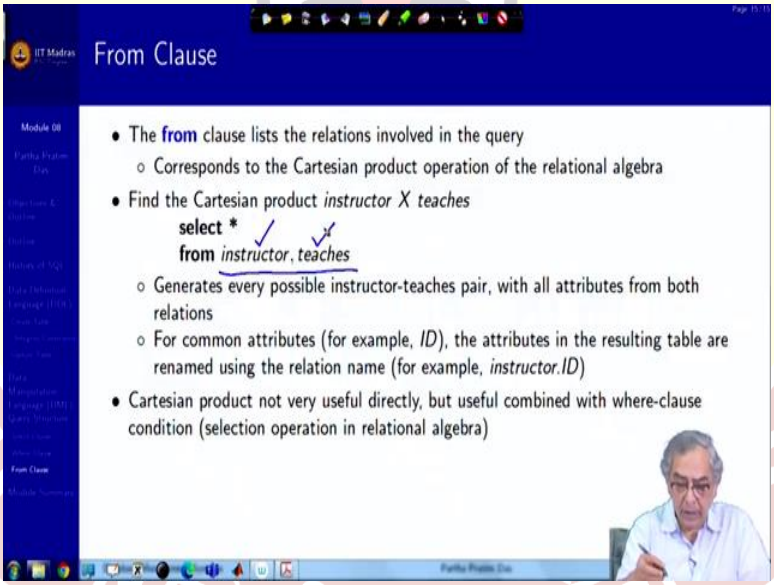




Now, the second part, the where clause is puts the condition or rather the third part, but in order of how you operate it comes at the second which is as I said is a Boolean condition. So, the result of this predicate is always true or false. And what is that you think about the query without the where clause, you have a result. So, you are taking all names of instructors.

And then on that, while you are taking this instead of taking all names, you are putting this condition that is you will not take all names from all tuples even select only those tuples where the department name is Computer Science. So, you are becoming selective. So, where clause basically is the selection predicate or the selection operation of the relational algebra like select clause is the projection operation, your where clause is the selection operation. So, it gives all conditions and you can naturally have any kind of compound condition. So, we say so, what will this do, it will have department name Computer Science, salary greater than 80,000 and names of those instructors will be only taken.

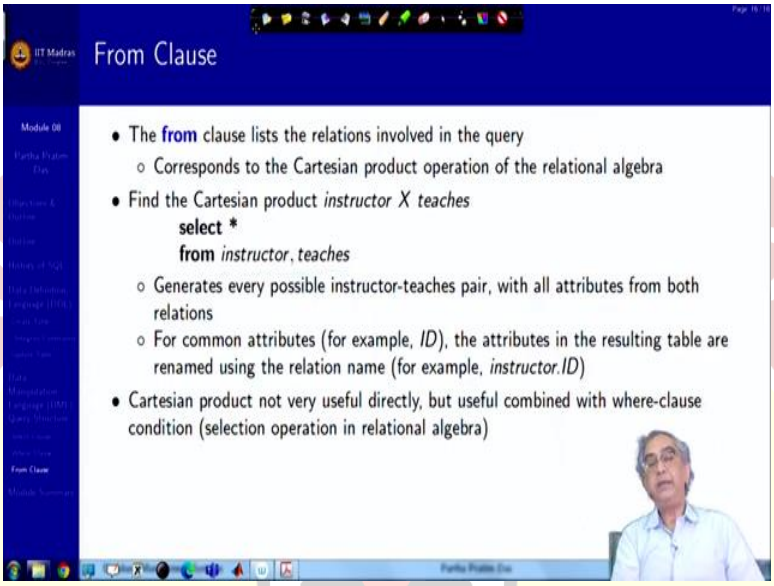
(Refer Slide Time: 27:46)

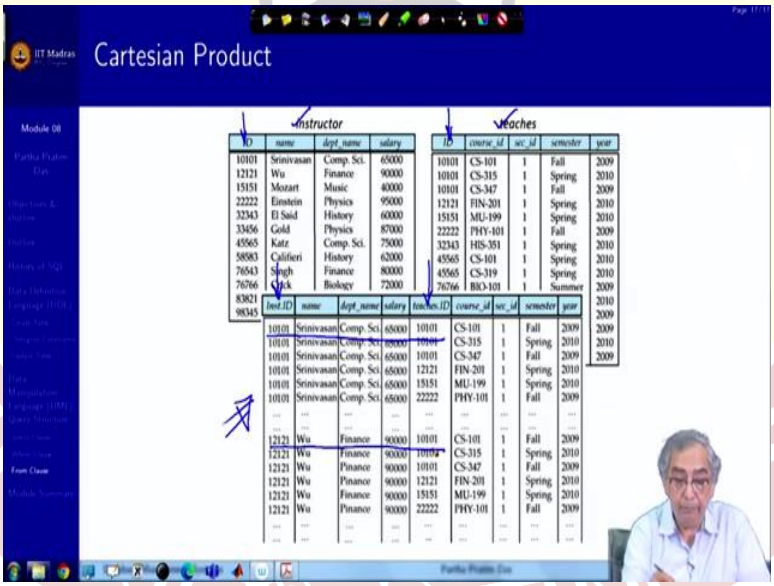


Finally, the from clause it tells you two things, one is it tells you the scope of which relations you are dealing with. And you could do from at least one table or you can have multiple tables, when you do multiple tables, then the thing that it gets is it by default does a Cartesian product. So, we are saying select *, what does that mean? That all attributes to be included. And the tables we are using are instructor and teachers. So, it will make a Cartesian product. What is Cartesian

product? Combination of all tuples. So, it will make a Cartesian product and do that selection and put everything in that table.

(Refer Slide Time: 28:41)

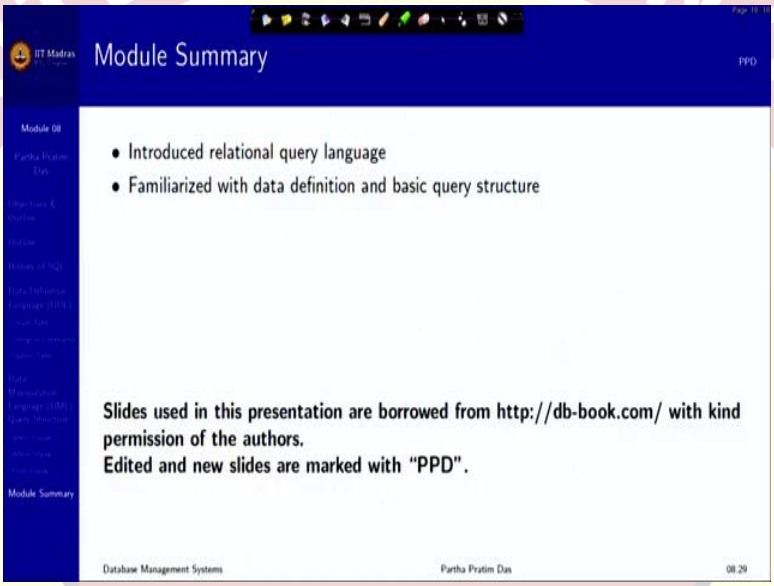




Now, the question certainly is do we need a Cartesian product? What does it mean and so on? We will come to all those, Cartesian product per say may not give you much value unless you have some specific *where* clause otherwise a Cartesian product is a big table like this, you have taken instructor, teaches and you have instructor here, teaches here and you have the result table which is a Cartesian product, all combinations.

But what you can do is I can see that there are two IDs. One is ID in the teaches and one is the ID in the instructor. Now, when they are equal, what does that mean? This is the details of the instructor and this is the details of the course that the instructor teaches, so that becomes meaningful. So, when I have this, it is meaningful, but when I have this, this is not meaningful. So, what can I do? I can simply put it where clause to make it simpler to make it shorter and meaningful that way.

(Refer Slide Time: 29:59)



So, that is what we will do and we will talk more about in the next module. So, in this we have just given the basic introduction, familiarization introduction to the DDL and the DML. And thank you for your attention and we will meet again in module 9 and take this forward.